

Análise e Projeto de Software Orientado a Objetos

Modelagem Conceitual: Entidades, Relacionamentos e Interfaces

Leonardo S. Victorio

Faculdade de Computação - UFMS

2026/1



- 1 Da Elicitação ao Modelo Conceitual
- 2 O Modelo Conceitual
- 3 Entidades, Atributos e Relacionamentos
- 4 Construindo o Modelo Conceitual
- 5 Do Modelo Conceitual às Interfaces
- 6 Conclusão

O que estudamos até aqui

Na aula anterior, vimos as etapas iniciais da engenharia de requisitos:

- **Elicitação:** entrevistas, etnografia, cenários e histórias de usuário.
- **Casos de uso:** descrição das interações entre atores e o sistema.
- **Diagrama de casos de uso:** visão gráfica de alto nível das funcionalidades.
- **Documento de Requisitos (SRS):** especificação formal do que o sistema deve fazer.
- **Validação:** revisões, prototipação e geração de casos de teste.

Questão central desta aula

Temos os requisitos. Temos os casos de uso. **E agora?** Como começamos a **estruturar** o sistema orientado a objetos?



O Caminho do Desenvolvimento Orientado a Objetos

A partir dos requisitos e casos de uso, seguimos um processo estruturado:

1. Levantar **requisitos** e documentar **casos de uso**.
2. Construir um **Modelo Conceitual** do domínio.
3. Identificar as **entidades principais** e seus relacionamentos.
4. Derivar as **interfaces de infraestrutura** (das entidades principais).
5. Derivar as **interfaces de aplicação** (dos casos de uso).
6. Definir a **arquitetura de componentes** do sistema.
7. Detalhar os componentes em **diagramas de classe e sequência**.

Foco desta aula

Passos **2, 3, 4 e 5**: construir o modelo conceitual e derivar as interfaces.

Por que o Modelo Conceitual?

Problema

- Casos de uso descrevem **o que** o sistema faz.
- Mas ainda não sabemos **com quais entidades** o sistema trabalha.
- Não há mapeamento direto de casos de uso para classes de implementação.

Solução: Modelo Conceitual

- Identifica as **entidades do domínio** relevantes ao sistema.
- Define **relacionamentos** entre essas entidades.
- Serve de **vocabulário comum** entre equipe técnica e cliente.
- Guia a **arquitetura** e o design de componentes.

Analogia

O modelo conceitual é como a **planta baixa** de um edifício: antes de construir, precisamos saber quais cômodos existem e como se conectam.

Modelagem Estrutural: Uma das Perspectivas

Ao modelar um sistema orientado a objetos, adotamos diferentes **perspectivas** complementares:

- Externa** Modela o contexto e o ambiente do sistema (diagrama de contexto, casos de uso).
- Interação** Modela como o sistema e os componentes interagem (diagramas de sequência, casos de uso).
- Estrutural** Modela a organização estática do sistema — **entidades, atributos e relacionamentos** (diagrama de classes, modelo conceitual).
- Comportamental** Modela o comportamento dinâmico em resposta a eventos (diagramas de estado, atividades).

Foco desta aula

A perspectiva **estrutural**: construir o modelo que representa as entidades do domínio e suas relações.

O que é um Modelo Conceitual?

Definição

Um **Modelo Conceitual** é uma representação gráfica das **entidades do domínio do problema** e dos **relacionamentos** entre elas, dentro do escopo do sistema a ser desenvolvido.

- É expresso como um **diagrama UML**, utilizando a notação de classes — sem métodos, apenas entidades e atributos essenciais.
- Representa o **domínio do problema**, não a solução de software.
- Entidades principais são marcadas com o estereótipo «main_entity».
- É construído **antes** do diagrama de classes de design.
- Serve de base para definir a **arquitetura de componentes** do sistema.



Modelo Conceitual vs. Diagrama de Classes

Modelo Conceitual

- Representa o **domínio** (mundo real).
- Sem métodos; atributos apenas quando essenciais.
- Entidades são **conceitos**, não classes de código.
- Usa estereótipos como «main_entity».
- Produzido na fase de **análise**.

Diagrama de Classes

- Representa a **solução** de software.
- Inclui atributos e **métodos**.
- Classes são **unidades de código**.
- Usa visibilidade (+, -, #).
- Produzido na fase de **design**.

Conclusão

O modelo conceitual é um **precursor** do diagrama de classes: define o vocabulário do domínio antes de detalhar a implementação.

O Domínio e o Escopo do Sistema

- O **domínio** de um sistema é o conjunto de todos os conceitos e entidades do mundo real relacionados ao problema.
- O **escopo** é a **fatia do domínio** que o sistema efetivamente cobre.
- Nem toda entidade do domínio precisa estar no modelo conceitual — apenas as que são **relevantes para o escopo**.

Exemplo: domínio hoteleiro

Hóspede, Quarto, Reserva,
Funcionário, Fornecedor,
Concorrente, Cidade, País,
Moeda...

Escopo: sistema de controle

Hóspede, Quarto, Reserva,
Funcionário, Conta — somente o
que o sistema gerencia.

Regra prática

Restrinja o domínio ao escopo definido pelos casos de uso antes de modelar.

Identificando Entidades do Domínio

O primeiro passo é levantar candidatos a entidades a partir dos **casos de uso** e do **enunciado do sistema**:

Bons candidatos

- Substantivos relevantes ao negócio.
- Conceitos que **armazenam dados** importantes.
- Entidades que **participam** de casos de uso.
- Conceitos com **identidade própria** no domínio.

Descartar

- Atributos simples (nome, data, valor).
- Conceitos **fora do escopo**.
- Duplicatas com nomes diferentes.
- Entidades de **implementação** (banco de dados, interface gráfica).

Dica prática

Leia cada caso de uso e sublinhe todos os **substantivos**. Eles são os candidatos iniciais.

Entidades Principais: o Estereótipo «main_entity»

Após listar os candidatos, precisamos distinguir as entidades **principais** das secundárias:

Definição

Uma **entidade principal** («main_entity») é uma entidade do domínio **independente** — ela existe por si mesma e outras entidades **dependem** dela para fazer sentido.

- As entidades principais são marcadas com o estereótipo «main_entity» no modelo conceitual.
- Cada entidade principal se tornará a **base de uma interface de infraestrutura**.
- Entidades que só existem em relação a outra são **secundárias** (ou dependentes).

Importância

A distinção correta entre entidades principais e secundárias é **fundamental** para derivar uma arquitetura coesa.

O Teste da Ausência

Para decidir se uma entidade é **principal** ou secundária, aplique o **Teste da Ausência**:

Pergunta

“Se esta entidade **não existisse**, as entidades associadas a ela ainda fariam sentido no sistema?”

Entidade Principal

Cliente — Se não houvesse clientes, não haveria Reservas, Contas nem Pagamentos.

⇒ *Resposta: NÃO* ⇒
«main_entity»

Entidade Secundária

Reserva — Se não houvesse reservas, ainda haveria Clientes, Cômodos e o Hotel funcionando.

⇒ *Resposta: SIM* ⇒ Entidade secundária

Atenção

O teste nem sempre é absoluto — use o bom senso e o contexto do **escopo do sistema** para decidir.

Tipos de Relacionamento

O modelo conceitual utiliza a notação UML para representar relacionamentos:

Associação Relação estrutural genérica entre entidades. Linha sólida simples.

Ex.: Cliente faz Reserva.

Agregação Relação “todo-parte” fraca: a parte pode existir sem o todo. Losango vazio.

Ex.: Hotel possui Cômodos (cômodo pode existir sem hóspede atual).

Composição Relação “todo-parte” forte: a parte não existe sem o todo. Losango preenchido.

Ex.: Conta contém Itens de despesa (item não existe fora da conta).

Generalização Herança: entidade filha herda características da entidade pai. Seta com triângulo.

Ex.: Funcionário ← Recepcionista, Camareira.



Multiplicidade nos Relacionamentos

A **multiplicidade** indica quantas instâncias de cada entidade participam do relacionamento:

Notação	Significado	Exemplo
1	Exatamente um	Um pedido tem 1 cliente
0..1	Zero ou um (opcional)	Um func. tem 0 ou 1 supervisor
1..*	Um ou mais	Um hotel tem 1 ou mais cômodos
*	Zero ou mais	Um cliente faz 0 ou mais reservas
2..4	Entre dois e quatro	Um time tem entre 2 e 4 jogadores

Regra

Sempre indique a multiplicidade **nos dois extremos** de cada associação. Ambiguidade na multiplicidade gera erros de implementação.

Processo de Construção: Visão Geral

Construir um modelo conceitual segue um processo iterativo de seis passos:

- 1 **Restringir o domínio ao escopo** definido pelos casos de uso.
- 2 **Listar candidatos a entidades** a partir dos substantivos dos casos de uso.
- 3 **Identificar as entidades principais** com o Teste da Ausência.
- 4 **Definir os relacionamentos** entre as entidades (e suas multiplicidades).
- 5 **Adicionar atributos essenciais** às entidades (apenas os que são conceitos do domínio).
- 6 **Revisar e simplificar** o modelo com o cliente e a equipe.

Processo iterativo

Não existe modelo conceitual perfeito na primeira tentativa. **Itere** com os stakeholders até que o modelo reflita com fidelidade o domínio do sistema.

Passo 1: Restringir o Domínio ao Escopo

- Revise os **casos de uso** levantados na etapa anterior.
- Para cada caso de uso, pergunte: “Quais entidades do domínio este caso de uso **manipula**?”
- Mantenha apenas as entidades que **aparecem diretamente** nos casos de uso ou são indispensáveis para eles.
- Descarte conceitos de domínio que estão **fora do escopo** do sistema.

Dentro do escopo

Entidades manipuladas diretamente pelo sistema:
Reserva, Conta, Cômodo, Cliente.

Fora do escopo

Entidades do domínio que o sistema não gerencia: Fornecedor de alimentos, Empresa de manutenção.

Passo 2 e 3: Identificar as Entidades Principais

Passo 2: Listar candidatos

Extraia todos os substantivos relevantes dos casos de uso e do enunciado do sistema. Agrupe sinônimos e elimine duplicatas.

Passo 3: Aplicar o Teste da Ausência

Para cada candidato, pergunte: *“Se esta entidade não existisse, as demais ainda fariam sentido?”*

- Resposta **NÃO** ⇒ Entidade **principal** («main_entity»).
- Resposta **SIM** ⇒ Entidade secundária (dependente de outra).

Resultado esperado

Uma lista de 4 a 8 entidades principais, que formarão o núcleo do modelo conceitual e servirão de base para a arquitetura.

Estudo de Caso: Sistema de Controle de Hotéis

Contexto: Um hotel precisa de um sistema para gerenciar suas operações. Os principais casos de uso levantados foram:

- Registrar reserva de cliente
- Registrar ocupação de cômodo
- Registrar saída (check-out) de cliente
- Registrar pagamento
- Lançar despesas na conta
- Emitir fatura/conta
- Cadastrar cômodos e tarifas
- Gerar relatórios gerenciais

Entidades candidatas (extraídas dos casos de uso)

Hotel, Cômodo, Tarifa, Cliente, Reserva, Ocupação, Conta, Despesa, Pagamento, Fatura, Funcionário, Controle Financeiro.



Estudo de Caso: Identificando as Entidades

Aplicando o Teste da Ausência ao sistema de hotel:

Ent. Principais «main_entity»

- **Hotel** — centraliza tudo
- **Cliente** — sem ele, nada faz sentido
- **Conta** — controla débitos/créditos
- **ControleFinanceiro** — controla transações
- **Funcionario** — opera o sistema

Entidades Secundárias

- **Cômodo** — parte do Hotel
- **Reserva** — depende de Cliente e Cômodo
- **Ocupacao** — depende de Reserva
- **Pagamento** — depende de Conta
- **Despesa** — depende de Conta
- **Tarifa** — depende de Cômodo

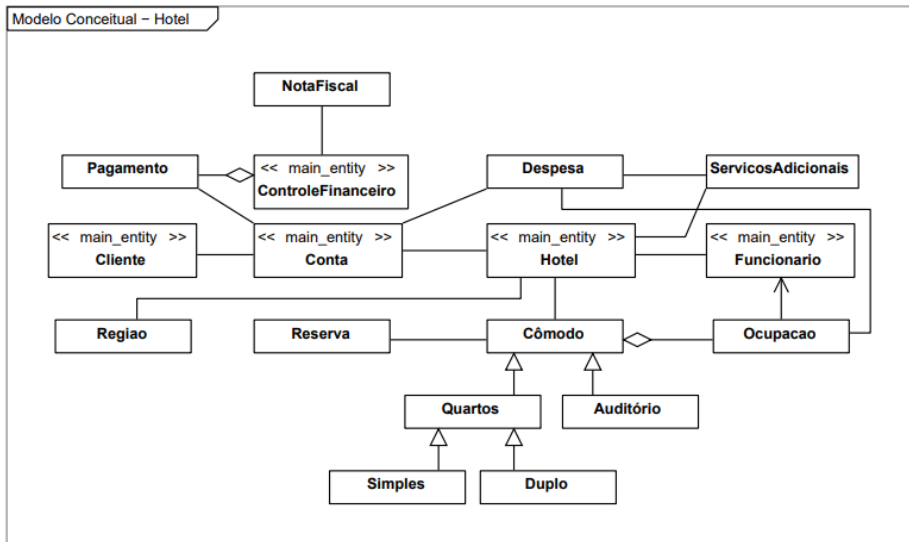
Estudo de Caso: Relacionamentos no Modelo

Os principais relacionamentos identificados entre as entidades:

- **Hotel** agrega **Cômodo** (1 hotel possui 1..* cômodos)
- **Hotel** emprega **Funcionario** (1 hotel emprega 1..* funcionários)
- **Cliente** realiza **Reserva** (1 cliente faz 0..* reservas)
- **Reserva** reserva **Cômodo** (1 reserva envolve 1..* cômodos)
- **Reserva** origina **Ocupacao** (1 reserva gera 0..1 ocupação)
- **Conta** registra **Despesa** (1 conta tem 0..* despesas)
- **Conta** registra **Pagamento** (1 conta tem 0..* pagamentos)
- **ControleFinanceiro** gerencia **Conta** (1 controle gerencia 1..* contas)
- **Cômodo** possui **Tarifa** (1 cômodo tem 1..* tarifas)



Modelo Conceitual



Estudo de Caso: Lendo o Modelo

Por que cada entidade é principal?

Hotel É o próprio contexto do sistema. Sem ele, cômodos, funcionários e reservas não têm onde existir.

Cliente É o ator central do negócio. Sem clientes, não há reservas, contas nem pagamentos.

Funcionario Opera o sistema e presta os serviços. Independe dos clientes para existir.

Conta Controla o relacionamento financeiro com cada cliente. Sem contas, pagamentos e despesas perdem contexto.

ControleFinanceiro Gerencia todas as transações financeiras do hotel. É independente e centraliza as contas.

Regra de ouro

Cada «main_entity» deve fazer sentido **isoladamente** no domínio do hotel.

Representação textual do modelo conceitual

Entidade	Estereótipo	Relacionamentos principais
Hotel	«main_entity»	agrega Cômodo; emprega Funcionario
Cliente	«main_entity»	realiza Reserva; possui Conta
Conta	«main_entity»	registra Despesa e Pagamento
ControleFin.	«main_entity»	gerencia Conta
Funcionario	«main_entity»	empregado por Hotel
Cômodo	(secundária)	parte de Hotel; reservado em Reserva
Reserva	(secundária)	feita por Cliente; origina Ocupacao
Ocupacao	(secundária)	originada de Reserva
Despesa	(secundária)	lançada em Conta
Pagamento	(secundária)	lançado em Conta
Tarifa	(secundária)	associada a Cômodo



A Ponte entre o Modelo Conceitual e a Arquitetura

O modelo conceitual não é um fim em si mesmo: ele alimenta **dois caminhos paralelos** rumo à arquitetura:

Caminho 1: via Entidades Principais

Cada «**main_entity**» do modelo conceitual origina uma **Interface de Infraestrutura**.

“O que o sistema precisa armazenar e gerenciar?”

Caminho 2: via Casos de Uso

Grupos de casos de uso relacionados originam **Interfaces de Aplicação**.

“O que o sistema precisa fazer?”

Resultado

As interfaces de infraestrutura e de aplicação definem os **contratos** que os componentes da arquitetura devem implementar.

Interfaces de Infraestrutura: das Entidades Principais

Para cada «main_entity» do modelo conceitual, define-se uma interface de infraestrutura:

Entidade P.	Int. de Infra.	Responsabilidade
ControleFin.	IFinMgr	Gerenciar transações financeiras
Conta	IContaMgr	Gerenciar contas dos clientes
Hotel	IHotelMgr	Gerenciar dados do hotel e cômodos
Cliente	IClienteMgr	Gerenciar cadastro de clientes
Funcionario	IFuncMgr	Gerenciar cadastro de funcionários

Convenção de nomenclatura

Interfaces de infraestrutura seguem o padrão I<NomeDaEntidade>Mgr, indicando que são **gerenciadores** da entidade correspondente.

Interfaces de Aplicação: dos Casos de Uso

Os casos de uso são agrupados por **funcionalidade** e cada grupo origina uma interface de aplicação:

Grupo de UC	Int. de Aplicação	Exemplos
Op. de Pagamento	I0pPagamento	Registrar pagamento, emitir r
Op. de Despesas	I0pDespesas	Lançar despesa, consultar co
Op. de Cômodos	I0pComodos	Reservar, registrar ocupação,
Op. de Cadastros	I0pCadastros	Cadastrar cliente, cômodo, fu

Convenção de nomenclatura

Interfaces de aplicação seguem o padrão I0p<Funcionalidade>, indicando que são **operações** oferecidas pelo sistema.

Interfaces de Aplicação e Infraestrutura: Diferenças

Infraestrutura

- Derivadas do **modelo conceitual**.
- Focadas em **persistência** e gerenciamento de dados.
- Refletem **entidades do domínio**.
- Focam em **acesso a dados**.
- Exemplo:
`IClienteMgr.buscarPorCPF()`.

Aplicação

- Derivadas dos **casos de uso**.
- Focadas em **lógica de negócio** e fluxos.
- Refletem **funcionalidades do sistema**.
- Implementadas por componentes de **negócio**.
- Exemplo:
`IOpComodos.registrarCheckIn`

Separação de responsabilidades

Essa divisão é a base do princípio de **separação entre lógica de negócio e acesso a dados**, fundamental em arquiteturas em camadas.

Interação entre as Perspectivas: Estrutural e de Interação

O modelo conceitual (perspectiva estrutural) e os diagramas de sequência (perspectiva de interação) se **complementam**:

- O modelo conceitual define **quais entidades** existem e como se relacionam estaticamente.
- Os diagramas de sequência mostram como essas entidades **colaboram dinamicamente** para realizar um caso de uso.
- Cada mensagem em um diagrama de sequência corresponde a um método de uma **interface**.

Exemplo: caso de uso “Registrar Reserva”

O diagrama de sequência mostraria as mensagens trocadas entre IOpComodos (interface de aplicação), IClienteMgr (infraestrutura) e IHotelMgr (infraestrutura) para realizar a reserva.



Do levantamento de requisitos à arquitetura de componentes

❶ Casos de Uso (requisitos funcionais do sistema)

↓ **Modelo Conceitual** (entidades do domínio no escopo)

↓ **Entidades Principais** (aplicando o Teste da Ausência)

Interfaces de Infraestrutura
(de cada «main_entity»)

Interfaces de Aplicação
(de grupos de casos de uso)

↓

↓ **Arquitetura de Componentes** (componentes que implementam as interfaces)

↓ **Diagramas de Classe e Sequência** (detalhamento de cada componente)



- O Modelo Conceitual representa as **entidades do domínio** e seus relacionamentos dentro do escopo do sistema.
- É expresso como um diagrama UML de classes — sem métodos, focado em **conceitos do negócio**.
- Entidades principais são marcadas com «**main_entity**» e identificadas pelo **Teste da Ausência**.
- O modelo deve ser **restrito ao escopo** definido pelos casos de uso, não ao domínio completo.
- É construído de forma **iterativa** com o cliente e a equipe técnica.

Pontos-Chave: Da Estrutura à Arquitetura

- As **entidades principais** do modelo conceitual geram as **interfaces de infraestrutura** (gerenciadores de dados).
- Os **grupos de casos de uso** geram as **interfaces de aplicação** (operações de negócio).
- Essa divisão implementa o princípio de **separação de responsabilidades**: lógica de negócio separada de acesso a dados.
- A perspectiva **estrutural** (modelo conceitual) e a perspectiva de **interação** (diagramas de sequência) são complementares.
- O fluxo *requisitos* → *modelo conceitual* → *interfaces* → *arquitetura* é a espinha dorsal do desenvolvimento orientado a objetos.



O que Vem a Seguir

- **Diagramas de Sequência:** mostram como os objetos colaboram para realizar cada caso de uso, usando as interfaces definidas.
- **Diagramas de Classes de Design:** detalham os componentes com atributos, métodos e visibilidade — evolução do modelo conceitual.
- **Arquitetura de Componentes:** define os módulos do sistema e como eles se comunicam via interfaces.
- **Padrões de Projeto:** soluções recorrentes para problemas de design que surgem ao detalhar os componentes.

Continuidade

O modelo conceitual construído nesta aula será o ponto de partida para os diagramas de sequência e de classes nas próximas aulas.



Referências Bibliográficas



SOMMERVILLE, Ian. *Engenharia de Software*. 10. ed. São Paulo: Pearson Education do Brasil, 2019. Cap. 5 (Modelagem de Sistema), seções 5.2 e 5.3.



BRITO, Kelvin S.; RUBIRA, Cecília M. F. *Desenvolvimento Baseado em Componentes com UML 2.0 e CCM*. Relatório Técnico IC-07-013. Campinas: IC-UNICAMP, 2007. Seção 3.2, Figuras 9 e 16.



CHEESMAN, John; DANIELS, John. *UML Components: A Simple Process for Specifying Component-Based Software*. Boston: Addison-Wesley, 2001.



BASS, Len; CLEMENTS, Paul; KAZMAN, Rick. *Software Architecture in Practice*. 3. ed. Boston: Addison-Wesley, 2012.



Dúvidas e Discussões

Leonardo S. Victorio

`leonardo.victorio@ufms.br`

Faculdade de Computação — UFMS

*“Um modelo é uma simplificação da realidade. Um modelo bem
construído é a base de toda boa arquitetura.”*

— Grady Booch

